

Spring Framework & Spring Boot Handbook

1. Why Spring Framework Was Introduced

Problems in traditional Java applications

- Manual object creation using the new keyword everywhere.
- Tight coupling between classes.
- Large Servlets containing request handling, business logic, and database code together.
- Repeated JDBC boilerplate code such as Connection, PreparedStatement, ResultSet, and close operations.
- Difficult testing, maintenance, and scaling.

```
// Traditional tight coupling
class Engine {
}

class Car {
    Engine engine = new Engine();
}
```

In the above example, Car is directly creating Engine. If Engine changes, Car is affected. This is tight coupling.

2. What is Spring Framework?

Spring Framework is a Java framework used to develop enterprise applications. It helps developers build loosely coupled, maintainable, testable, and scalable applications.

Main things Spring provides

- IoC Container to create and manage objects.
- Dependency Injection to provide required objects automatically.
- Spring MVC to build web applications.
- Spring JDBC and Spring Data JPA for database operations.
- Support for REST APIs, transactions, security, and microservices.

Traditional Java	Spring Framework
Manual object creation	Spring manages objects
Tight coupling	Loose coupling
Repeated boilerplate	Cleaner reusable code
Manual dependency handling	Dependency Injection
Difficult testing	Easy testing using injected dependencies

3. Spring Core Concepts

3.1 IoC - Inversion of Control

Definition: IoC means the control of object creation and object management is transferred from the programmer to the Spring Container.

In simple words: Instead of creating objects manually, we allow Spring to create and manage them.

```
// Without Spring
Engine engine = new Engine();
```

```
// With Spring
@Component
class Engine {
}

@Component
class Car {
    private Engine engine;

    public Car(Engine engine) {
        this.engine = engine;
    }
}
```

Here, Spring creates Engine and injects it into Car. The programmer does not manually create the Engine object.

3.2 IoC Container

Definition: IoC Container is the core part of Spring that creates objects, stores them, manages their lifecycle, and injects dependencies.

Responsibilities of IoC Container

- Scan classes marked with annotations like @Component, @Service, @Repository, and @Controller.
- Create bean objects.
- Inject dependencies between beans.
- Manage bean lifecycle from creation to destruction.

```
Spring starts
↓
Component scanning happens
↓
Beans are created
↓
Dependencies are injected
↓
Application becomes ready
```

3.3 Spring Bean

Definition: Any object created and managed by the Spring Container is called a Spring Bean.

```
@Component
public class Engine {
}
```

The Engine object created by Spring is called a bean.

3.4 Bean Lifecycle

Bean lifecycle means the different stages a bean goes through inside the Spring Container.

```
Spring Container starts
↓
Bean object is created
↓
Dependencies are injected
↓
@PostConstruct method runs
↓
Bean is ready to use
↓
@PreDestroy method runs before shutdown

@Component
public class Engine {

    public Engine() {
```

```

        System.out.println("Constructor Called");
    }

    @PostConstruct
    public void init() {
        System.out.println("Bean Initialized");
    }

    @PreDestroy
    public void destroy() {
        System.out.println("Bean Destroyed");
    }
}

```

3.5 Dependency Injection - DI

Definition: Dependency Injection means Spring provides the required object to a class from outside instead of the class creating it manually.

Dependency means one class needs another class. Injection means giving that dependency from outside.

Constructor Injection - Recommended

```

@Service
public class StudentService {

    private final StudentRepository repository;

    public StudentService(StudentRepository repository) {
        this.repository = repository;
    }
}

```

- Best for mandatory dependencies.
- Easy to test.
- Makes dependencies clear.
- Supports final fields and immutability.

Setter Injection

```

@Service
public class StudentService {

    private StudentRepository repository;

    @Autowired
    public void setRepository(StudentRepository repository) {
        this.repository = repository;
    }
}

```

Setter injection is useful when a dependency is optional.

Field Injection - Not Preferred

```

@Service
public class StudentService {

    @Autowired
    private StudentRepository repository;
}

```

- Harder to test because dependency is hidden.
- Uses reflection internally.
- Does not clearly show required dependencies.
- Avoid in real industry code when possible.

3.6 Important Spring Stereotype Annotations

Annotation	Meaning	Layer
@Component	Generic Spring-managed bean	General utility/helper classes
@Service	Business logic class	Service layer
@Repository	Database access class	Repository/DAO layer
@Controller	Handles web requests and returns views	Web layer
@RestController	Handles REST requests and returns data directly	API layer

Note: @Service, @Repository, and @Controller are specialized forms of @Component. They give meaning to the class based on its responsibility.

4. Spring MVC

Definition: Spring MVC is a web framework inside Spring that follows the Model-View-Controller design pattern.

MVC Meaning

Part	Meaning	Responsibility
Model	Data holder	Carries data from Controller to View
View	UI page	Displays data to the user, such as JSP
Controller	Request handler	Receives browser requests and returns response/view

Servlet/JSP flow vs Spring MVC flow

Servlet/JSP	Spring MVC
Browser -> Servlet -> JSP	Browser -> DispatcherServlet -> Controller -> JSP
request.setAttribute()	model.addAttribute()
RequestDispatcher forward()	return view name
doGet()/doPost()	@GetMapping/@PostMapping

5. DispatcherServlet - Most Important Spring MVC Concept

Definition: DispatcherServlet is the front controller of Spring MVC. It receives every web request first and then dispatches that request to the correct controller method.

Why DispatcherServlet is needed

In old Servlet applications, every URL could be mapped to a separate Servlet. In Spring MVC, one central servlet receives all requests and routes them properly. This central servlet is DispatcherServlet.

```
Browser Request
↓
DispatcherServlet
↓
HandlerMapping finds correct controller method
↓
```

```
Controller method executes
↓
Model data is prepared
↓
ViewResolver finds JSP
↓
JSP renders response
↓
Browser gets final HTML
```

DispatcherServlet Step-by-Step Flow

1. User sends request from browser, for example: `http://localhost:8080/home`
2. Embedded Tomcat receives the request.
3. DispatcherServlet receives the request first.
4. HandlerMapping checks which controller method matches `/home`.
5. Controller method executes business flow and adds data into Model.
6. Controller returns a logical view name, for example `"home"`.
7. ViewResolver converts `"home"` into `/WEB-INF/jsp/home.jsp`.
8. JSP page is rendered into HTML and sent back to browser.

Example

```
@Controller
public class HomeController {

    @GetMapping("/home")
    public String home(Model model) {
        model.addAttribute("name", "Guru");
        return "home";
    }
}
```

If JSP prefix is `/WEB-INF/jsp/` and suffix is `.jsp`, then return `"home"` means Spring will open `/WEB-INF/jsp/home.jsp`.

6. Model and addAttribute

Definition: Model is used to send data from Controller to View page such as JSP.

```
model.addAttribute("name", "Guru");
```

This is similar to Servlet code:

```
request.setAttribute("name", "Guru");
```

In JSP, we can read this data using Expression Language:

```
<h1>Welcome ${name}</h1>
```

7. @Controller vs @RestController

@Controller

@Controller is used when the response is usually a JSP/HTML page. It works with ViewResolver.

```
@Controller
public class HomeController {

    @GetMapping("/home")
    public String home() {
        return "home";
    }
}
```

Here, return `"home"` means open `home.jsp`.

@RestController

@RestController is used to build REST APIs. It returns data directly, usually JSON or plain text. It does not return JSP pages.

```
@RestController
public class StudentRestController {

    @GetMapping("/api/student")
    public Student getStudent() {
        return new Student(1, "Rahul", "Java");
    }
}
```

@Controller	@RestController
Used for MVC web pages	Used for REST APIs
Returns JSP/view name	Returns data directly
Needs ViewResolver for JSP	No ViewResolver needed
Common with JSP/Thymeleaf	Common with Angular/React/Postman APIs

8. Spring Boot

Definition: Spring Boot is built on top of Spring Framework. It simplifies Spring application development by reducing manual configuration.

What Spring Boot provides

- Auto configuration.
- Embedded server such as Tomcat.
- Starter dependencies.
- Easy application.properties configuration.
- Faster project setup.

@SpringBootApplication

@SpringBootApplication is the main annotation used on the startup class of a Spring Boot application.

It internally combines three important annotations:

Annotation	Purpose
@Configuration	Marks the class as a configuration class. It can define Spring beans.
@EnableAutoConfiguration	Automatically configures Spring Boot based on dependencies present in pom.xml.
@ComponentScan	Scans the current package and sub-packages for Spring components.

```
@SpringBootApplication
public class DemoApplication {

    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);
    }
}
```

How a Spring Boot application runs

1. main() method starts.
2. SpringApplication.run() executes.
3. Spring ApplicationContext/IOC Container starts.
4. Component scanning happens from the main class package.
5. Beans are created and dependencies are injected.
6. Auto configuration checks dependencies and configures required components.
7. Embedded Tomcat server starts.
8. DispatcherServlet is initialized for Spring MVC requests.
9. Application becomes ready to handle browser/API requests.

9. Creating a Spring Boot Project Using Spring Starter Project

Using STS/Eclipse

1. Open STS or Eclipse.
2. Go to File -> New -> Spring Starter Project.
3. Enter project name, for example student-management.
4. Select Maven as project type.
5. Select Java version.
6. Enter Group, for example com.example.
7. Enter Artifact, for example student-management.
8. Choose packaging as Jar for normal Spring Boot applications.
9. Add required dependencies.
10. Click Finish and wait for Maven to download dependencies.

Common dependencies for Spring Boot MVC with JSP and DB

- Spring Web - required for MVC and web applications.
- Tomcat Jasper - required for JSP compilation.
- JSTL - required for JSTL tags in JSP pages.
- Spring Boot Starter JDBC - required for Spring JDBC/JdbcTemplate.
- Spring Data JPA - required for JPA/Hibernate.
- MySQL Driver - required to connect MySQL database.
- Spring Boot DevTools - useful for development auto restart.

10. application.properties

Definition: application.properties is the main configuration file in Spring Boot. It is created inside src/main/resources. Spring Boot reads this file at application startup and applies the configurations.

Where to create it

```
src/main/resources/application.properties
```

Common configurations

```
# Change default server port
server.port=9090

# Add application context path
server.servlet.context-path=/student-app
```

```
# JSP View Resolver configuration
spring.mvc.view.prefix=/WEB-INF/jsp/
spring.mvc.view.suffix=.jsp

# MySQL database configuration
spring.datasource.url=jdbc:mysql://localhost:3306/studentdb
spring.datasource.username=root
spring.datasource.password=root
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# JPA/Hibernate configuration
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

How to change default port

By default, Spring Boot runs on port 8080. To change it, add this in application.properties:

```
server.port=9090
```

Now the application will run on:

```
http://localhost:9090
```

How to configure JSP view path

When a controller returns "home", Spring needs to know where home.jsp is located. This is done using prefix and suffix.

```
spring.mvc.view.prefix=/WEB-INF/jsp/
spring.mvc.view.suffix=.jsp
```

Now this controller return value:

```
return "home";
```

will be resolved as:

```
/WEB-INF/jsp/home.jsp
```

How to configure database connection

```
spring.datasource.url=jdbc:mysql://localhost:3306/studentdb
spring.datasource.username=root
spring.datasource.password=root
```

These properties tell Spring Boot which database to connect with and which username/password to use.

11. Spring Boot with JSP

Where to create JSP files

In Spring Boot, JSP files should be created under src/main/webapp/WEB-INF/jsp.

```
src/main/webapp/WEB-INF/jsp/home.jsp
src/main/webapp/WEB-INF/jsp/addStudent.jsp
src/main/webapp/WEB-INF/jsp/viewStudents.jsp
```

Why JSP files are stored inside WEB-INF

WEB-INF is a protected folder. Files inside WEB-INF cannot be accessed directly from the browser.

This is good because users should not directly open JSP pages. They should come through Controller.

```
Correct flow:
Browser -> Controller -> JSP
```

```
Wrong flow:
Browser -> Direct JSP access
```

Example: If home.jsp is inside WEB-INF, the user cannot directly access it using:


```
http://localhost:8080/WEB-INF/jsp/home.jsp
```

Instead, user must access:

```
http://localhost:8080/home
```

Then Controller returns the JSP page.

Required dependencies for JSP

```
<!-- Spring Web -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>

<!-- JSP Support -->
<dependency>
    <groupId>org.apache.tomcat.embed</groupId>
    <artifactId>tomcat-embed-jasper</artifactId>
</dependency>

<!-- JSTL -->
<dependency>
    <groupId>jakarta.servlet.jsp.jstl</groupId>
    <artifactId>jakarta.servlet.jsp.jstl-api</artifactId>
</dependency>

<dependency>
    <groupId>org.glassfish.web</groupId>
    <artifactId>jakarta.servlet.jsp.jstl</artifactId>
</dependency>
```

Spring Boot MVC JSP example

```
@Controller
public class HomeController {

    @GetMapping("/home")
    public String home(Model model) {
        model.addAttribute("name", "Guru");
        return "home";
    }
}

<!-- /WEB-INF/jsp/home.jsp -->
<html>
<body>
    <h1>Welcome ${name}</h1>
</body>
</html>
```

12. Spring JDBC

Definition: Spring JDBC is a Spring module that simplifies traditional JDBC programming using JdbcTemplate.

Traditional JDBC problem

```
Connection con = DriverManager.getConnection(url, username, password);
PreparedStatement ps = con.prepareStatement("insert into student(name) values(?)");
ps.setString(1, "Guru");
ps.executeUpdate();
ps.close();
con.close();
```

This contains repeated boilerplate code. Spring JDBC reduces this using JdbcTemplate.

JdbcTemplate

Definition: JdbcTemplate is a Spring class used to execute SQL queries without manually writing connection closing logic.

```
jdbcTemplate.update(
    "insert into student(name, course) values(?, ?)",
    student.getName(),
    student.getCourse()
);
```

Repository example using JdbcTemplate

```
@Repository
public class StudentRepository {

    private final JdbcTemplate jdbcTemplate;

    public StudentRepository(JdbcTemplate jdbcTemplate) {
        this.jdbcTemplate = jdbcTemplate;
    }

    public void saveStudent(Student student) {
        String sql = "insert into student(name, course) values(?, ?)";
        jdbcTemplate.update(sql, student.getName(), student.getCourse());
    }

    public List<Student> getAllStudents() {
        String sql = "select * from student";
        return jdbcTemplate.query(sql, (rs, rowNum) -> {
            Student s = new Student();
            s.setId(rs.getInt("id"));
            s.setName(rs.getString("name"));
            s.setCourse(rs.getString("course"));
            return s;
        });
    }
}
```

Important JdbcTemplate methods

Method	Use
update()	INSERT, UPDATE, DELETE
query()	Fetch multiple rows
queryForObject()	Fetch one row or one value
execute()	Execute DDL or general SQL

13. Spring Data JPA

Definition: Spring Data JPA simplifies database operations by reducing manual SQL for common CRUD operations.

```
@Entity
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    private String name;
    private String course;
}

public interface StudentRepository extends JpaRepository<Student, Integer> {
}
```

JpaRepository automatically provides methods like save(), findAll(), findById(), deleteById(), and more.

14. Standard Spring Boot MVC Project Structure

```
src/main/java
  com.example.studentapp
    StudentAppApplication.java
    controller
      StudentController.java
    service
      StudentService.java
    repository
      StudentRepository.java
    entity
      Student.java
    dto
      StudentDTO.java

src/main/resources
  application.properties

src/main/webapp/WEB-INF/jsp
  addStudent.jsp
  viewStudents.jsp
  updateStudent.jsp
```

Layer responsibility

Layer	Responsibility
Controller	Handles browser/API request and response
Service	Contains business logic
Repository	Handles database operations
Entity	Represents database table
DTO	Transfers data between layers/UI/API
JSP	Displays UI to the user

15. Best Practices

- Use constructor injection instead of field injection.
- Keep business logic in Service layer, not Controller.
- Never write database code inside JSP.
- Keep JSP files inside WEB-INF/jsp and access them only through Controller.
- Use application.properties for configuration instead of hardcoding values.
- Use @Controller for JSP pages and @RestController for REST APIs.
- Use Spring JDBC when you want manual SQL control.
- Use Spring Data JPA when you want faster CRUD development.

16. Student Practice Exercises

1. Create a Spring Boot MVC project using Spring Starter Project.
2. Configure server.port=9090 in application.properties.
3. Create /WEB-INF/jsp/home.jsp and return it from HomeController.
4. Create a Student form using JSP.
5. Send form data to Controller using @PostMapping.

6. Display form data in another JSP using Model.
7. Configure MySQL database connection in application.properties.
8. Use JdbcTemplate to insert student data.
9. Display all students using JSTL c:forEach.
10. Convert the same project from Spring JDBC to Spring Data JPA.

17. Important Interview Questions

- What is Spring Framework?
- Why was Spring introduced?
- What is IoC?
- What is IoC Container?
- What is a Spring Bean?
- Explain bean lifecycle.
- What is Dependency Injection?
- Why is constructor injection preferred?
- Difference between @Component, @Service, @Repository, and @Controller.
- What is Spring MVC?
- What is DispatcherServlet?
- Explain Spring MVC request flow.
- What is Model in Spring MVC?
- Difference between @Controller and @RestController.
- What is @SpringBootApplication?
- Which three annotations are inside @SpringBootApplication?
- How does a Spring Boot application run?
- What is application.properties?
- How do you change the default port in Spring Boot?
- Where do we create JSP files in Spring Boot?
- Why do we store JSP files inside WEB-INF?
- What is JdbcTemplate?
- Difference between Spring JDBC and Spring Data JPA.

18. Final Summary

Spring Framework helps Java developers build loosely coupled applications using IoC and Dependency Injection.

Spring MVC organizes web applications using Controller, Model, ViewResolver, JSP, and DispatcherServlet.

Spring Boot reduces manual configuration using auto configuration, starter dependencies, embedded server, and application.properties.

Spring JDBC simplifies traditional JDBC through JdbcTemplate.

Spring Data JPA further simplifies database operations by providing ready-made CRUD methods.